



**UNIVERSIDAD CATÓLICA DE SANTA MARÍA**  
**ESCUELA PROFESIONAL DE INGENIERÍA DE SISTEMAS**



**LENGUAJES DE PROGRAMACIÓN III - 04**

**PRÁCTICA N.º 05: GENERICS EN JAVA**

| <b>CÓDIGO</b> | <b>APELLIDOS Y NOMBRES</b>       | <b>FECHA</b> |
|---------------|----------------------------------|--------------|
| 2025000074    | LLERENA QUISPE, JHOAOO SEBASTIÁN | 30/09/2026   |
| 2025001757    | MEDINA BARRIOS, GUSTAVO DAVID    | 30/09/2026   |
| 2025002328    | MONTES YATO, MAURICIO GONZALO    | 30/09/2026   |

## **1. RECONOCIMIENTOS**

Expresamos nuestro reconocimiento a la Universidad Católica de Santa María y a los docentes de Lenguajes de Programación III por la orientación brindada en el estudio de Java. La práctica permite relacionar los conceptos de tipos, igualdad y colecciones con implementaciones concretas, contrastar el comportamiento de los programas y documentar sus resultados.

## **2. RESUMEN**

El informe desarrolla las cuatro actividades y los cuatro ejercicios propuestos de la Guía 05. Se implementan métodos genéricos para imprimir arreglos y comparar objetos, una pila con búsqueda y comparación sin alterar su estado, una clase Par y un Contenedor aplicado a un inventario de equipos de redes. Se responden las siete preguntas del cuestionario con apoyo de documentación oficial. Los 16 archivos Java de implementación y el archivo de pruebas se compilaron con Java 17, sin advertencias, y superaron 57 verificaciones. Cada ejercicio incluye explicación, evidencia visual y enlaces al código. Se documentan además las imprecisiones identificadas en la guía.

**Palabras clave:** Java, genéricos, parámetros de tipo, pila, igualdad, ArrayList, wildcard, Comparable.

## ÍNDICE

|                                                     |    |
|-----------------------------------------------------|----|
| 1. Reconocimientos                                  | 1  |
| 2. Resumen                                          | 1  |
| 3. Introducción                                     | 3  |
| 4.1 Actividad 1. Impresión de arreglos e intervalos | 4  |
| 4.2 Actividad 2. Pila genérica y búsqueda contains  | 6  |
| 4.3 Actividad 3. Igualdad genérica y valores null   | 8  |
| 4.4 Actividad 4. Comparación de dos pilas           | 10 |
| 5.1 Ejercicio propuesto 1. Clase Par con dos tipos  | 12 |
| 5.2 Ejercicio propuesto 2. Igualdad entre pares     | 14 |
| 5.3 Ejercicio propuesto 3. Método imprimirPar       | 16 |
| 5.4 Ejercicio propuesto 4. Contenedor de inventario | 18 |
| 6. Cuestionario                                     | 20 |
| 7. Conclusiones                                     | 23 |
| 8. Referencias bibliográficas                       | 24 |
| 9. Anexos y observaciones de la guía                | 26 |

Los enlaces azules del documento son clicables. Cada apartado enlaza a su código fuente en el mismo repositorio de las prácticas anteriores.

### 3. INTRODUCCIÓN

Los genéricos permiten expresar clases y operaciones reutilizables que conservan información sobre los tipos de datos utilizados. En esta práctica se pasa de un método que imprime arreglos a estructuras que almacenan y comparan elementos. El desarrollo sigue el orden de la Guía 05 y conserva la organización general del informe de la Práctica 03 (Guevara et al., s. f.; Llerena Quispe et al., 2026).

La implementación distingue los parámetros de una clase, como E en Pila<E>, de los parámetros que un método declara por sí mismo, como <F, S> en imprimirPar. Los tipos concretos se determinan al utilizar cada estructura o invocar cada método. Así se reduce la duplicación de código sin reemplazar las validaciones que cada problema necesita.

#### 3.1 Alcance y organización

Las actividades abordan intervalos válidos, funcionamiento LIFO, igualdad y tratamiento de null. Los ejercicios propuestos construyen progresivamente Par y Contenedor. Se emplean clases pequeñas, arreglos y ArrayList para mantener un nivel adecuado al curso. Los datos son demostrativos y el almacenamiento permanece en memoria.

El código se organiza en los paquetes actividades, propuestos y cuestionario. Pila se comparte entre las actividades 2 y 4. Par se comparte entre los cuatro ejercicios propuestos. Las clases de prueba tienen su propio main para ejecutar cada apartado de manera independiente.

#### 3.2 Repositorio y evidencia

[https://github.com/jhoao0/javaUcsm/tree/main/practica\\_05](https://github.com/jhoao0/javaUcsm/tree/main/practica_05)

Las figuras de código representan los archivos fuente de esta entrega. Las figuras de ejecución se generan a partir de la salida real guardada en archivos de texto, no de una simulación de resultados ni de un IDE. Los enlaces permiten consultar la implementación completa, incluidas las clases auxiliares que no aparecen íntegramente en las figuras.

Entorno de comprobación: OpenJDK 17.0.20. Compilación con UTF-8, -Xlint:all y -Werror. Resultado: 57/57 verificaciones correctas. Las instrucciones para reproducir las ejecuciones se encuentran en el anexo y en README.md.

## 4. ACTIVIDADES

### 4.1 Actividad 1. Impresión de arreglos e intervalos

Se implementa el método genérico imprimirArreglo(E[]) y una sobrecarga que recibe los índices inferior y superior. La segunda versión imprime un intervalo inclusivo, devuelve la cantidad de elementos mostrados y lanza `InvalidSubscriptException` cuando el intervalo incumple las condiciones de la guía (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/ImprimirArreglo.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/ImprimirArreglo.java)

**Figura 1.** Implementación de Impresión de arreglos e intervalos.

Código fuente | actividades/ImprimirArreglo.java | líneas 1-26

```
package actividades;

import java.util.Objects;

public class ImprimirArreglo {
    public static <E> void imprimirArreglo(E[] arreglo) {
        Objects.requireNonNull(arreglo, "El arreglo no puede ser null");
        for (E elemento : arreglo) System.out.print(elemento + " ");
        System.out.println();
    }

    /** Imprime el intervalo inclusivo y devuelve su cantidad de elementos. */
    public static <E> int imprimirArreglo(E[] arreglo, int inferior,
                                         int superior) {
        Objects.requireNonNull(arreglo, "El arreglo no puede ser null");
        if (inferior < 0 || superior < 0 || inferior >= arreglo.length
            || superior >= arreglo.length || superior <= inferior) {
            throw new InvalidSubscriptException(
                "Intervalo inválido: [" + inferior + ", " + superior + "]");
        }
        for (int i = inferior; i <= superior; i++) {
            System.out.print(arreglo[i] + " ");
        }
        System.out.println();
        return superior - inferior + 1;
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

El parámetro E representa el tipo de referencia de los elementos. La misma lógica recibe `Integer[]`, `Double[]` y `Character[]`. La declaración `<E>` aparece antes del tipo de retorno. La sobrecarga se reconoce por sus dos argumentos `int` adicionales, mientras que la versión original solo recibe el arreglo (Oracle, s. f.-e).

Antes de recorrer el arreglo se comprueban ambos extremos: ninguno puede ser negativo ni alcanzar la longitud del arreglo. Además, superior debe ser estrictamente mayor que inferior. La comprobación se realiza antes de imprimir, por lo que una entrada inválida no produce una salida parcial.

## 4.1 Actividad 1. Impresión de arreglos e intervalos (continuación)

**Figura 2.** Ejecución verificada del apartado.

Salida capturada | Java 17 | ImprimirArreglo

```
Integer completo:
1 2 3 4 5 6
Intervalo [1, 3]:
2 3 4
Elementos impresos: 3
Double completo:
1.1 2.2 3.3 4.4 5.5 6.6 7.7
Intervalo [1, 3]:
2.2 3.3 4.4
Elementos impresos: 3
Character completo:
H O L A
Intervalo [1, 3]:
O L A
Elementos impresos: 3
InvalidSubscriptException: Intervalo inválido: [-1, 2]
InvalidSubscriptException: Intervalo inválido: [0, 6]
InvalidSubscriptException: Intervalo inválido: [2, 2]
InvalidSubscriptException: Intervalo inválido: [3, 1]
```

Fuente: elaboración propia. Salida real capturada con Java 17.

El recorrido utiliza  $i \leq \text{superior}$ . Por ello,  $[1, 3]$  selecciona tres posiciones y el retorno se calcula como  $\text{superior} - \text{inferior} + 1$ . En el arreglo de enteros se imprimen 2, 3 y 4. En el de caracteres se imprimen O, L y A. El límite superior está incluido.

La ejecución utiliza ambas versiones con los tres arreglos indicados en la guía. Cada intervalo válido devuelve 3. Los casos  $[-1, 2]$ ,  $[0, 6]$ ,  $[2, 2]$  y  $[3, 1]$  generan la excepción solicitada, que el programa captura para continuar con las demás demostraciones.

La condición  $\text{superior} \leq \text{inferior}$  excluye los intervalos de un único elemento. Se mantiene porque está expresamente indicada en el enunciado. La referencia null al arreglo se rechaza por separado mediante `requireNonNull`. Un arreglo vacío puede imprimirse completo, pero no tiene un intervalo válido para la sobrecarga.

La impresión de  $k$  elementos cuesta  $O(k)$  y utiliza  $O(1)$  de memoria auxiliar. La validación de índices tiene costo constante. Las pruebas también revisan que el contenido impreso coincida exactamente con el intervalo, además de comprobar la cantidad devuelta.

## 4.2 Actividad 2. Pila genérica y búsqueda contains

Se programa una pila de capacidad fija basada en el ejemplo de la guía y se incorpora contains(E elemento). La búsqueda debe comenzar en el tope, avanzar hacia el fondo y devolver un booleano sin retirar ni reinsertar elementos (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/Pila.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/Pila.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/PruebaPila.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/PruebaPila.java)

**Figura 3.** Implementación de Pila genérica y búsqueda contains.

Código fuente | actividades/Pila.java

```
/** Pila LIFO de capacidad fija. Admite null como elemento. */
public class Pila<E> {
    private final E[] elementos;
    private int superior = -1;

    public Pila() { this(10); }

    @SuppressWarnings("unchecked")
    public Pila(int capacidad) {
        int tamaño = capacidad > 0 ? capacidad : 10;
        // El arreglo permanece privado y solo recibe valores de tipo E.
        elementos = (E[]) new Object[tamaño];
    }

    public void push(E valor) {
        if (superior == elementos.length - 1) {
            throw new ExcepcionPilaLlena("No se puede insertar: " + valor);
        }
        elementos[++superior] = valor;
    }

    public E pop() {
        if (superior == -1) {
            throw new ExcepcionPilaVacía("No se puede extraer de una pila vacía");
        }
        E valor = elementos[superior];
        elementos[superior--] = null; // Libera la referencia retirada.
        return valor;
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

La variable superior comienza en -1. push comprueba si hay espacio y después incrementa el índice para guardar el valor. pop comprueba si la pila está vacía, recupera el elemento superior, limpia su celda y disminuye el índice. El método size devuelve superior + 1.

## 4.2 Actividad 2. Pila genérica y búsqueda contains (continuación)

**Figura 4.** Búsqueda desde el tope, sin modificar la pila.

Código fuente | actividades/Pila.java

```
public int size() { return superior + 1; }
public int capacidad() { return elementos.length; }

/** Busca desde el tope hacia el fondo, sin retirar elementos. */
public boolean contains(E elemento) {
    for (int i = superior; i >= 0; i--) {
        if (Objects.equals(elementos[i], elemento)) return true;
    }
    return false;
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

**Figura 5.** Ejecución verificada del apartado.

Salida capturada | Java 17 | PruebaPila

```
Pila [fondo -> tope]: [10, 20, 30]
contains(20): true
contains(99): false
Después de buscar: [10, 20, 30]; tamaño=3
ExcepcionPilaLlena: No se puede insertar: 40
Extracción LIFO: 30, 20, 10
ExcepcionPilaVacía: No se puede extraer de una pila vacía
Pila String: [Java, null]
contains(null): true
Pila Double: [1.1, 2.2]
```

Fuente: elaboración propia. Salida real capturada con Java 17.

La creación del arreglo requiere una conversión no comprobada de `Object[]` a `E[]`. Se conserva la estrategia de la guía y la supresión de advertencia se limita al constructor. El arreglo permanece privado, recibe únicamente `E` mediante `push` y nunca se devuelve al exterior. No se crea `new E[]`, operación no permitida (Oracle, s. f.-i).

`contains` recorre únicamente las posiciones activas, desde `superior` hasta 0. `Objects.equals` permite encontrar `null` sin intentar ejecutar un método sobre una referencia nula. Si localiza una coincidencia, retorna `true` de inmediato. Si termina el recorrido, retorna `false` (Oracle, s. f.-h).

La salida muestra que buscar 20 y 99 no cambia `[10, 20, 30]` ni su tamaño. La extracción posterior produce 30, 20 y 10, lo que comprueba el orden LIFO. También se verifican las excepciones por exceso de capacidad y extracción en vacío, así como pilas de `String` y `Double`.

Una búsqueda cuesta  $O(n)$  en el peor caso y  $O(1)$  de memoria auxiliar. `push` y `pop` cuestan  $O(1)$ . La capacidad inválida se sustituye por 10, conforme al constructor de la guía. El comentario original de `push` se corrige: su firma `void` no devuelve `true`.

### 4.3 Actividad 3. Igualdad genérica y valores null

Se desarrolla IgualGenerico con el método esIgualA. La comparación se realiza mediante equals y se prueba con valores primitivos, Integer, String, Object y null. Se conserva una versión literal adicional para evidenciar qué sucede cuando se invoca equals sobre null (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/IgualGenerico.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/IgualGenerico.java)

**Figura 6.** Implementación de Igualdad genérica y valores null.

Código fuente | actividades/IgualGenerico.java | líneas 1-12

```
package actividades;

public class IgualGenerico {
    /** Versión literal: falla si el primer argumento es null. */
    public static <T> boolean esIgualALiteral(T primero, T segundo) {
        return primero.equals(segundo);
    }

    /** Compara mediante equals y trata explícitamente el caso null. */
    public static <T> boolean esIgualA(T primero, T segundo) {
        return primero == null ? segundo == null : primero.equals(segundo);
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

La versión literal devuelve primero.equals(segundo). Compila, pero si primero es null lanza NullPointerException. La versión segura pregunta primero si la primera referencia es null: en ese caso solo devuelve true si la segunda también lo es. Cuando la primera referencia existe, delega la comparación a equals.

El método no obliga a todos los objetos a compartir un criterio de igualdad. Object utiliza identidad por defecto, mientras que String compara contenido. Por eso dos objetos Object recién creados son distintos, pero dos cadenas construidas por separado con el texto Java resultan iguales (Oracle, s. f.-g).

Las llamadas con int, double, char y boolean son válidas porque sus valores se convierten en Integer, Double, Character y Boolean mediante autoboxing. El parámetro de tipo sigue siendo de referencia. Esto no equivale a escribir una clase genérica parametrizada con un tipo primitivo (Oracle, s. f.-b).

### 4.3 Actividad 3. Igualdad genérica y valores null (continuación)

**Figura 7.** Ejecución verificada del apartado.

Salida capturada | Java 17 | IgualGenerico

```
int 5 y 5 (autoboxing): true
double 2.5 y 2.5: true
char A y A: true
boolean true y false: false
Integer 1000 y 1000: true
String de igual contenido: true
Misma referencia Object: true
Dos Object diferentes: false
null y null: true
null y Java: false
Java y null: false
Integer 5 y Double 5.0: false
Versión literal con null: NullPointerException
```

Fuente: elaboración propia. Salida real capturada con Java 17.

Los casos null/null, null/Java y Java/null producen true, false y false en la versión segura. La versión literal genera la excepción esperada cuando el primer argumento es null. El bloque try-catch la muestra sin interrumpir el programa.

Integer 5 y Double 5.0 producen false: aunque representan magnitudes numéricas equivalentes, sus métodos equals no los consideran objetos iguales. Un parámetro T común tampoco garantiza que los dos argumentos tengan la misma clase concreta, porque la inferencia puede encontrar un supertipo común.

El resultado de la actividad depende del contrato de equals de cada clase. Los genéricos reutilizan la operación y conservan la información de tipos, pero no sustituyen una definición correcta de igualdad. La complejidad depende de equals: comparar cadenas puede requerir recorrer su contenido.

## 4.4 Actividad 4. Comparación de dos pilas

Se agrega `esIgual(Pila<E> otraPila)` a la misma clase `Pila`. Dos pilas se consideran iguales cuando tienen la misma cantidad de elementos y los mismos valores en las posiciones correspondientes. La operación conserva ambas estructuras (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/Pila.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/Pila.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/actividades/PruebaIgualdadPilas.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/actividades/PruebaIgualdadPilas.java)

**Figura 8.** Implementación de Comparación de dos pilas.

Código fuente | actividades/Pila.java | líneas 46-57

```
/** Compara cantidad y orden lógico, no la capacidad disponible. */
public boolean esIgual(Pila<E> otraPila) {
    if (otraPila == null || size() != otraPila.size()) return false;
    for (int i = superior; i >= 0; i--) {
        if (!Objects.equals(elementos[i], otraPila.elementos[i])) {
            return false;
        }
    }
    return true;
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

Primero se descarta una referencia nula y se comparan los tamaños lógicos. Luego se recorren las posiciones desde el tope hasta el fondo. Una sola diferencia basta para retornar `false`. Si todas las comparaciones coinciden, el método devuelve `true`.

No se usa `pop` para comparar. Leer directamente los arreglos privados desde la propia clase evita vaciar las pilas o necesitar pilas auxiliares. Es válido que una instancia de `Pila` acceda a los campos privados de otra instancia de la misma clase.

El tamaño lógico es el número de valores almacenados, no la capacidad reservada. Por ello una pila con capacidad 3 y otra con capacidad 8 pueden ser iguales si ambas contienen [10, 20, 30]. Dos pilas vacías también son iguales, cualquiera que sea su capacidad.

La firma utiliza `E`, que ya fue declarado por la clase `Pila<E>`. Técnicamente es un método de una clase genérica que usa ese parámetro, no un método que introduce su propio `<T>`. Se mantiene la firma pedida por la guía.

## 4.4 Actividad 4. Comparación de dos pilas (continuación)

**Figura 9.** Ejecución verificada del apartado.

Salida capturada | Java 17 | PruebalgualdadPilas

```
A: [10, 20, 30]; capacidad=3
B: [10, 20, 30]; capacidad=8
Mismos valores y orden: true
Orden distinto: false
Tamaño distinto: false
Comparación con null: false
Dos pilas vacías: true
Pilas con null: true
Después: A=[10, 20, 30]; B=[10, 20, 30]
Tope conservado A/B: 30/30
```

Fuente: elaboración propia. Salida real capturada con Java 17.

La comparación devuelve true para A y B, false para una pila con orden invertido y false para una con menos elementos. La comparación con null también devuelve false. Los valores null dentro de ambas pilas se comparan de manera segura.

La última parte de la ejecución vuelve a mostrar A y B y extrae sus topes: ambas devuelven 30. Esto comprueba que esIgual no alteró los valores ni el orden. Las pruebas automatizadas verifican además que los tamaños siguen siendo los originales.

La comparación tiene costo  $O(n)$  si se recorren todos los elementos y  $O(1)$  de memoria auxiliar. Con tamaños distintos termina inmediatamente. Se asume que equals de cada elemento funciona correctamente. La pila no realiza una copia profunda ni redefine la igualdad de los objetos que contiene.

## 5. EJERCICIOS PROPUESTOS

### 5.1 Ejercicio propuesto 1. Clase Par con dos tipos

Se crea `Par<F, S>` con dos atributos, constructor, getters, setters y una representación textual con el formato solicitado: (Primero: x, Segundo: y). Los parámetros `F` y `S` permiten que cada componente tenga un tipo independiente (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Par.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Par.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/PruebaAccesoresPar.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/PruebaAccesoresPar.java)

**Figura 10.** Implementación de Clase Par con dos tipos.

Código fuente | propuestos/Par.java | líneas 1-18

```
package propuestos;

import java.util.Objects;

/** Par ordenado con tipos independientes para sus dos componentes. */
public class Par<F, S> {
    private F primero;
    private S segundo;

    public Par(F primero, S segundo) {
        this.primero = primero;
        this.segundo = segundo;
    }

    public F getPrimero() { return primero; }
    public S getSegundo() { return segundo; }
    public void setPrimero(F primero) { this.primero = primero; }
    public void setSegundo(S segundo) { this.segundo = segundo; }
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

`F` describe el primer componente y `S` el segundo. En `Par<String, Integer>`, `getPrimero` devuelve `String` y `getSegundo` devuelve `Integer` sin conversiones explícitas. Los setters aceptan únicamente los valores compatibles con cada posición.

El constructor inicializa ambos componentes. Los campos son privados para concentrar el acceso y la modificación en los métodos de la clase. `toString` construye la representación solicitada, por lo que imprimir el objeto permite observar sus dos valores.

La prueba crea el par (Java, 17), consulta los getters y después actualiza ambos componentes. El resultado final representa el curso y la sesión: (Lenguajes de Programación III, 5). La misma estructura puede emplearse con otros tipos sin duplicar la clase.

## 5.1 Ejercicio propuesto 1. Clase Par con dos tipos (continuación)

**Figura 11.** Ejecución verificada del apartado.

Salida capturada | Java 17 | PruebaAccesoresPar

```
Par inicial: (Primero: Java, Segundo: 17)
getPrimero(): Java
getSegundo(): 17
Después de los setters: (Primero: Lenguajes de Programación III, Segundo: 5)
```

Fuente: elaboración propia. Salida real capturada con Java 17.

La captura permite comprobar el estado inicial, los valores recuperados y el estado posterior a las modificaciones. Cambiar el contenido no cambia los tipos fijados al declarar el objeto: `setSegundo` continúa recibiendo `Integer` durante toda la vida de ese par.

Los getters y setters tienen costo  $O(1)$ . La conversión textual depende de la representación de los componentes. Se admite `null` y la concatenación lo muestra como el texto `null`, lo que evita una llamada directa a `toString` sobre una referencia nula.

La clase del repositorio incluye también `esIgual`, incorporado por el ejercicio siguiente. Se conserva una sola implementación de `Par` para que todos los programas utilicen la misma definición y para evitar copias divergentes del código.

## 5.2 Ejercicio propuesto 2. Igualdad entre pares

Se agrega `esIgual` a `Par` y se comprueba su comportamiento mediante `PruebaPar`. Deben coincidir el primer componente con el primero del otro par y el segundo con el segundo, manteniendo el orden establecido (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Par.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Par.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/PruebaPar.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/PruebaPar.java)

**Figura 12.** Implementación de Igualdad entre pares.

Código fuente | `propuestos/Par.java` | líneas 19-29

```
public boolean esIgual(Par<F, S> otro) {
    return otro != null && Objects.equals(primer, otro.primer)
        && Objects.equals(segundo, otro(segundo));
}

@Override
public String toString() {
    return "(Primer: " + primer + ", Segundo: " + segundo + ")";
}
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

La comparación `primer` descarta que el otro par sea `null`. Después exige dos coincidencias mediante el operador lógico `&&`. `Objects.equals` permite comparar los componentes cuando alguno de ellos es `null` y delega en `equals` cuando corresponde (Oracle, s. f.-h).

No basta con que los mismos valores aparezcan en cualquier posición. Un `Par<String, String>` con (A, B) es distinto de otro con (B, A). La relación que representa un par es ordenada, por lo que intercambiar los componentes cambia su significado.

`esIgual` es un método propio solicitado por el enunciado. No sobrescribe `Object.equals` porque tiene otro nombre y otra firma. En consecuencia, no se debe suponer que las colecciones utilizarán `esIgual` automáticamente para buscar estos pares.

## 5.2 Ejercicio propuesto 2. Igualdad entre pares (continuación)

**Figura 13.** Ejecución verificada del apartado.

Salida capturada | Java 17 | PruebaPar

```
A: (Primero: Java, Segundo: 17)
B: (Primero: Java, Segundo: 17)
Mismos valores: true
Segundo distinto: false
Otro par null: false
Componentes null iguales: true
Valores invertidos: false
```

Fuente: elaboración propia. Salida real capturada con Java 17.

PruebaPar compara dos pares con cadenas de igual contenido, modifica el segundo valor de uno de ellos y repite la comparación. Se observa true antes del cambio y false después. También se verifica un par de referencia null, componentes null coincidentes y componentes invertidos.

Para componentes simples la comparación requiere dos operaciones de igualdad. El costo real depende de la implementación de equals de F y S. La comparación no modifica atributos y no necesita objetos auxiliares.

Si se quisiera utilizar igualdad por contenido con HashSet o HashMap, habría que definir equals(Object) y hashCode de forma coherente. Esa ampliación no es necesaria para este ejercicio, que pide expresamente esIgual. La clase Persona del siguiente ejercicio sí muestra ambos métodos.

### 5.3 Ejercicio propuesto 3. Método imprimirPar

Se implementa en Main el método estático genérico imprimirPar(Par<F, S> par). Se llama con los tipos String/Integer, Double/Boolean y Persona/Integer, exactamente como establece el ejercicio (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Main.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Main.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Persona.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Persona.java)

**Figura 14.** Implementación de Método imprimirPar.

Código fuente | propuestos/Main.java | líneas 1-22

```
package propuestos;

public class Main {
    public static <F, S> void imprimirPar(Par<F, S> par) {
        System.out.println(par);
    }

    public static void main(String[] args) {
        Par<String, Integer> curso = new Par<>("Java", 17);
        Par<Double, Boolean> medida = new Par<>(3.5, true);
        Par<Persona, Integer> persona = new Par<>(new Persona("Ana", 20), 101);
        imprimirPar(curso);
        imprimirPar(medida);
        imprimirPar(persona);
        Par<Persona, Integer> copia = new Par<>(new Persona("Ana", 20), 101);
        System.out.println("Personas distintas, mismos datos: "
            + persona.esIgual(copia));
        System.out.println("hashCode coherente: "
            + (persona.getPrimero().hashCode()
            == copia.getPrimero().hashCode()));
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

El método declara sus propios parámetros <F, S> antes de void. Aunque Main no es una clase genérica, cada invocación infiere los tipos a partir del par recibido. System.out.println utiliza la representación que proporciona Par.toString (Oracle, s. f.-e).

El primer ejemplo relaciona Java con 17. El segundo combina una medida de 3.5 con true. El tercero vincula una persona llamada Ana con el identificador 101. Son datos demostrativos creados para la práctica, no registros reales.

### 5.3 Ejercicio propuesto 3. Método imprimirPar (continuación)

**Figura 15.** Ejecución verificada del apartado.

Salida capturada | Java 17 | Main

```
(Primero: Java, Segundo: 17)
(Primero: 3.5, Segundo: true)
(Primero: Ana (20 años), Segundo: 101)
Personas distintas, mismos datos: true
hashCode coherente: true
```

Fuente: elaboración propia. Salida real capturada con Java 17.

Persona define toString para que la salida muestre nombre y edad. Además, implementa igualdad por ambos atributos y un hashCode coherente. De esta manera se puede comprobar que dos instancias distintas con los mismos datos también se consideran iguales al comparar los pares.

Las tres líneas iniciales muestran que un solo método imprime los tres pares. La prueba adicional compara personas distintas con nombre y edad coincidentes y obtiene true. También confirma que sus códigos hash son iguales.

La clase Persona es final e inmutable. Esta decisión mantiene sencilla la comparación y evita que una subclase cambie el criterio de igualdad. Objects.equals protege el nombre si es null. La implementación respeta la exigencia de que objetos iguales tengan el mismo hashCode (Oracle, s. f.-g).

## 5.4 Ejercicio propuesto 4. Contenedor de inventario

Se elige como temática un inventario de equipos de un laboratorio de redes. Contenedor<String, Integer> relaciona el nombre del equipo con su cantidad y almacena múltiples Par<F, S> dentro de un ArrayList, conforme a los cuatro métodos exigidos (Guevara et al., s. f.).

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Contenedor.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Contenedor.java)

Programa de prueba / clase auxiliar:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/PruebaContenedor.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/PruebaContenedor.java)

**Figura 16.** Implementación de Contenedor de inventario.

Código fuente | propuestos/Contenedor.java | líneas 1-23

```
package propuestos;

import java.util.ArrayList;

/** Almacena pares y protege la estructura de la lista interna. */
public class Contenedor<F, S> {
    private final ArrayList<Par<F, S>> pares = new ArrayList<>();

    public void agregarPar(F primero, S segundo) {
        pares.add(new Par<>(primero, segundo));
    }

    public Par<F, S> obtenerPar(int indice) { return pares.get(indice); }

    public ArrayList<Par<F, S>> obtenerTodosLosPares() {
        // Copia superficial: otra lista que contiene los mismos objetos Par.
        return new ArrayList<>(pares);
    }

    public void mostrarPares() {
        for (Par<F, S> par : pares) System.out.println(par);
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

agregarPar construye un nuevo Par y lo añade a la lista. obtenerPar usa un índice para recuperar un registro. obtenerTodosLosPares entrega todos los registros en una nueva lista. mostrarPares recorre e imprime los pares almacenados.

ArrayList permite crecer al agregar elementos y acceder por índice. En la solución se declara ArrayList<Par<F, S>>, por lo que se conserva la información de ambos tipos. Los índices comienzan en 0 y un acceso inválido genera IndexOutOfBoundsException (Oracle, s. f.-a).

## 5.4 Ejercicio propuesto 4. Contenedor de inventario (continuación)

**Figura 17.** Ejecución verificada del apartado.

Salida capturada | Java 17 | PruebaContenedor

```
Inventario del laboratorio:
(Primero: Router, Segundo: 4)
(Primero: Switch, Segundo: 8)
(Primero: Punto de acceso, Segundo: 6)
obtenerPar(1): (Primero: Switch, Segundo: 8)
Switch actualizado: (Primero: Switch, Segundo: 10)
Total de pares devueltos: 3
Pares internos después de limpiar copia: 3
Índice 8: IndexOutOfBoundsException controlada
```

Fuente: elaboración propia. Salida real capturada con Java 17.

La lista devuelta es una copia superficial: su estructura es independiente, pero contiene los mismos objetos Par. Por eso limpiar esa lista no borra el inventario, mientras que modificar un Par obtenido del contenedor sí actualiza el registro correspondiente. Esta decisión se documenta para evitar confundirla con una copia profunda.

El programa registra 4 routers, 8 switches y 6 puntos de acceso. Luego recupera el índice 1, actualiza la cantidad de switches a 10 y obtiene los tres pares. Al limpiar la lista copiada, el contenedor conserva sus tres registros. Finalmente se controla un acceso al índice 8.

La demostración ejecuta los cuatro métodos solicitados y también una modificación mediante el setter de Par. El inventario se mantiene en memoria. La práctica no requiere archivos, base de datos ni interfaz gráfica.

Agregar tiene costo amortizado  $O(1)$  y obtener por índice cuesta  $O(1)$ . Copiar o mostrar todos los pares cuesta  $O(n)$ . La copia de la lista requiere  $O(n)$  de memoria adicional, mientras que la impresión recorre los registros sin construir otra colección (Oracle, s. f.-a).

## 6. CUESTIONARIO

### 1. ¿Qué significa restringir un tipo genérico con extends?

Una restricción establece qué tipos pueden sustituir al parámetro genérico. En `<T extends Number>`, `T` debe ser `Number` o una subclase. El cuerpo puede utilizar operaciones disponibles en `Number`, por ejemplo `doubleValue`. La palabra `extends` también se emplea cuando el límite es una interfaz (Oracle, s. f.-c).

En el ejemplo de esta entrega, `duplicar(T numero)` devuelve `numero.doubleValue() * 2`. La llamada `duplicar(7)` devuelve 14.0 y `duplicar(2.5)` devolvería 5.0. Un `String` no cumple el límite, por lo que `duplicar("Java")` no compilaría. El límite permite expresar la operación válida sin comprobar manualmente la clase de cada argumento.

### 2. ¿Cuáles son las limitaciones de los genéricos?

Los argumentos de tipo deben ser tipos de referencia. Por ello `Par<int, double>` es inválido y se utiliza `Par<Integer, Double>`. Tampoco se puede crear `new T()` ni `new T[n]`, ni declarar un campo `static` cuyo tipo sea el `T` de la clase. El borrado de tipos también impide distinguir `List<String>` de `List<Integer>` mediante una comprobación general en ejecución (Oracle, s. f.-i).

En la pila, estas limitaciones explican por qué el arreglo se crea como `Object[]` y su conversión se mantiene encapsulada. No se debe resolver el problema usando tipos sin parametrizar, porque eso pierde comprobaciones del compilador. El autoboxing de valores individuales facilita las llamadas con literales, pero no transforma un `int[]` existente en `Integer[]` (Oracle, s. f.-b).

## 6. CUESTIONARIO (continuación)

### 3. Caso de uso real y beneficios

Un inventario de laboratorio necesita relacionar cada equipo con una cantidad. `Contenedor<String, Integer>` expresa esa relación de forma explícita. Al recuperar el segundo componente se obtiene `Integer` y se pueden realizar operaciones numéricas sin convertir desde `Object`. Esta aplicación corresponde al ejercicio propuesto 4.

El beneficio se observa al modificar registros: el compilador rechazaría asignar "muchos" como cantidad mediante `setSegundo`. La misma clase puede reutilizarse con `Persona` e `Integer` para representar préstamos por usuario. La reutilización corresponde a la estructura, mientras que las reglas del negocio, como prohibir cantidades negativas, requerirían validaciones adicionales. El ejemplo de la práctica demuestra los genéricos y no pretende ser un inventario completo de producción.

### 4. Genéricos y jerarquías de clases

Aunque `Integer` hereda de `Number`, `List<Integer>` no hereda de `List<Number>`. Si esa asignación estuviera permitida, se podría introducir un `Double` en una lista destinada a enteros. Esta invariancia es una diferencia importante frente a pensar únicamente en la jerarquía de los tipos de elementos (Oracle, s. f.-f).

Para una operación de lectura numérica se puede recibir `List<? extends Number>`. El método `sumar` de la entrega acepta `List<Integer>` y procesa sus elementos como `Number`. No puede insertar arbitrariamente un `Integer` o `Double` en esa lista de tipo desconocido. Para recibir enteros en una lista destino resulta apropiado `List<? super Integer>`. Además, una clase puede extender una clase genérica conservando su parámetro o fijándolo a un tipo concreto.

### 5. ¿Qué es un wildcard?

El wildcard se representa mediante `?` y significa un tipo desconocido dentro de un tipo parametrizado. `List<?>` puede referirse a listas con distintos tipos de elementos. No equivale a `List<Object>`, porque esta última sí fija su parámetro en `Object` (Oracle, s. f.-j).

Las variantes principales son `?`, `? extends Tipo` y `? super Tipo`. La primera sirve cuando no se necesita conocer el tipo concreto. La segunda permite trabajar con un límite superior y la tercera establece un límite inferior. En `mostrar(List<?> lista)` basta recorrer cada elemento como `Object` para imprimirlo. No es necesario declarar un parámetro `T` que no se utilizará en otra parte de la firma.

## 6. CUESTIONARIO (continuación)

### 6. Diferencia entre T y ?

T es un parámetro con nombre que permite relacionar varias posiciones de una firma. En `<T> T primero(List<T> lista)`, el tipo del resultado queda vinculado al tipo de elementos de la lista. Así, una `List<Integer>` produce un `Integer` sin conversión explícita.

El símbolo `?` expresa un tipo desconocido y resulta útil cuando basta consumir una estructura sin devolver ese mismo tipo específico. `mostrar(List<?>)` puede imprimir listas diversas. Si primero recibiera `List<?>` y devolviera `Object`, el llamador perdería la información concreta del resultado. La elección depende de si es necesario expresar una relación entre tipos, no de que una sintaxis sea siempre superior a la otra (Oracle, s. f.-e, s. f.-j).

### 7. Significado de `<T extends Comparable<T>>`

La declaración limita T a un tipo que implemente `Comparable<T>`. Así, el método puede invocar `compareTo` para ordenar o seleccionar elementos del mismo tipo. El resultado de `compareTo` es negativo, cero o positivo según el orden relativo, y no tiene que ser exactamente -1 o 1 (Oracle, s. f.-d).

El ejemplo `maximo(T a, T b)` devuelve a cuando `a.compareTo(b)` es mayor o igual a cero, y b en caso contrario. Se comprueba con enteros y cadenas. El código asume argumentos no nulos. Para jerarquías en las que la comparación está definida en un supertipo puede resultar más flexible `<T extends Comparable<? super T>>`. La versión de la práctica conserva la forma solicitada en la pregunta.

Ejemplos compilados del cuestionario:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/cuestionario/EjemplosGenericos.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/cuestionario/EjemplosGenericos.java)

## 7. CONCLUSIONES

1. Los métodos genéricos permitieron imprimir arreglos y operar con pares de diferentes tipos mediante una misma implementación. La declaración de tipos hace explícitas las relaciones entre argumentos, atributos y resultados.
2. La validación previa de índices evita impresiones parciales e impide acceder a posiciones inexistentes. La restricción que excluye intervalos de un solo elemento se mantuvo para respetar el enunciado.
3. La pila conserva su estado durante contains y esIgual porque ambas operaciones leen las posiciones activas sin ejecutar push o pop. Las pruebas comprobaron tamaño, orden y valores después de las consultas.
4. La igualdad depende de la implementación de equals de cada tipo. El tratamiento explícito de null permite comparar de forma segura, y Persona muestra la relación necesaria entre equals y hashCode.
5. Contenedor reutiliza Par dentro de un ArrayList y demuestra los cuatro métodos solicitados. La copia superficial de la lista protege su estructura, pero conserva las referencias a los pares, una decisión que debe quedar documentada.
6. La compilación sin advertencias y las 57 verificaciones respaldan los comportamientos evaluados. Las evidencias y los enlaces directos permiten reproducir las ejecuciones y contrastar el informe con el código publicado.

### 7.1 Correspondencia con la rúbrica

**Métodos genéricos:** imprimirArreglo, esIgualA, imprimirPar y maximo.

**Clases genéricas:** Pila, Par y Contenedor.

**Igualdad y comparación:** esIgual, Persona.equals/hashCode y Comparable.

**ArrayList:** almacenamiento, acceso, recorrido y copia de pares.

**Documentación:** comentarios, README, evidencias, citas y explicaciones.

Los criterios de puntualidad y trabajo en equipo dependen de la entrega y de la participación real de los integrantes. El informe no atribuye tareas individuales que no hayan sido confirmadas.

## 8. REFERENCIAS BIBLIOGRÁFICAS

Guevara, K., Montesinos, Á., Santillana, M., Iquira, D., Galarza, M., & Villaverde, N. (s. f.). *Sesión N.º 05: Generics en Java* [Guía de prácticas y rúbrica de Lenguajes de Programación III]. Universidad Católica de Santa María.

Archivo proporcionado: Guía05\_LP3\_2026\_rub(1).pdf

Llerena Quispe, J. S., Medina Barrios, G. D., & Montes Yato, M. G. (2026). *Práctica N.º 03: POO avanzada y principios SOLID* [Informe académico]. Universidad Católica de Santa María.

Archivo proporcionado: Practica\_03\_LP3(1).pdf

Oracle. (s. f.-a). *ArrayList*. Java Platform, Standard Edition 17 API Specification.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/ArrayList.html>

Oracle. (s. f.-b). *Autoboxing and unboxing*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/data/autoboxing.html>

Oracle. (s. f.-c). *Bounded type parameters*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/generics/bounded.html>

Oracle. (s. f.-d). *Comparable*. Java Platform, Standard Edition 17 API Specification.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Comparable.html>

## 8. REFERENCIAS BIBLIOGRÁFICAS (continuación)

Oracle. (s. f.-e). *Generic methods*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/generics/methods.html>

Oracle. (s. f.-f). *Generics, inheritance, and subtypes*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/generics/inheritance.html>

Oracle. (s. f.-g). *Object*. Java Platform, Standard Edition 17 API Specification.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/lang/Object.html>

Oracle. (s. f.-h). *Objects*. Java Platform, Standard Edition 17 API Specification.

<https://docs.oracle.com/en/java/javase/17/docs/api/java.base/java/util/Objects.html>

Oracle. (s. f.-i). *Restrictions on generics*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/generics/restrictions.html>

Oracle. (s. f.-j). *Wildcards*. The Java Tutorials.

<https://docs.oracle.com/javase/tutorial/java/generics/wildcards.html>

Nota. Se emplean citas de autor y fecha y referencias conforme al estilo APA 7. Las letras distinguen las páginas de Oracle sin fecha. Los tutoriales consultados corresponden al material de JDK 8 y se contrastan con la API de Java 17 utilizada para la compilación.

## 9. ANEXOS

### 9.1 Observaciones sobre la Guía 05

Las páginas indicadas corresponden al orden del PDF proporcionado. Las observaciones siguientes no cambian el objetivo de los ejercicios.

**Tipos primitivos en la introducción (p. 1).** Se menciona `int` y `double` al describir el uso genérico. Como argumentos de tipo deben utilizarse `Integer` y `Double`. El propio material posteriormente sí distingue los tipos de referencia (Oracle, s. f.-i).

**Autoboxing de arreglos (p. 9).** La explicación indica que los arreglos primitivos se convierten automáticamente. La conversión automática corresponde a valores individuales. Un `int[]` no se convierte por sí solo en `Integer[]` (Oracle, s. f.-b).

**Retorno de push (pp. 6 y 12).** El comentario anuncia verdadero, pero la firma es `void`. La operación termina normalmente o lanza `ExcepcionPilaLlena`. Se corrige el comentario, conservando la firma.

**Igualdad de Persona (pp. 10-11 y rúbrica).** El material usa sobrecargar donde corresponde sobrescribir `equals(Object)`. El ejemplo omite `hashCode` y llama a `nombre.equals` sin proteger un nombre `null`. La entrega incorpora ambas correcciones (Oracle, s. f.-g).

**Actividad 3 y null (p. 13).** La comparación literal falla cuando el primer argumento es `null`. Se muestra la excepción y se añade una versión segura. Es un resultado que debe explicarse, no ocultarse.

**Terminología de la actividad 4 (p. 13).** `esIgual(Pila<E>)` utiliza el parámetro de la clase. No introduce uno propio como un método declarado con `<T>`. La firma solicitada sigue siendo válida.

**Numeración del documento (p. 13).** Ejercicios propuestos y cuestionario repiten V. El informe utiliza una secuencia propia coherente para sus secciones.

**Restricción de la actividad 1 (p. 12).** `superior <= inferior` se considera inválido. Esto impide intervalos de un elemento. Se trata como una condición expresa del enunciado y se respeta.

## 9.2 Reproducción y comprobación

1. Descargar el repositorio y abrir la carpeta practica\_05. Instalar un JDK 17 que incluya javac y java. No se necesitan dependencias externas.
2. En Windows ejecutar ejecutar.bat. En Linux o macOS ejecutar sh ejecutar.sh. Ambos scripts compilan el proyecto y lanzan las demostraciones y las pruebas.
3. Para ejecutar un solo apartado, compilar primero y luego utilizar el nombre completo de su clase, incluido el paquete. Ejemplo:

```
javac -encoding UTF-8 -Xlint:all -Werror -d out @sources.txt
java -cp out actividades.ImprimirArreglo
java -cp out Verificacion
```

**Figura 18.** Registro del entorno y de la compilación.

| Salida capturada   Java 17   compilacion                                                                                        |
|---------------------------------------------------------------------------------------------------------------------------------|
| Compilador: javac 17.0.20<br>Opciones: -encoding UTF-8 -Xlint:all -Werror<br>Resultado: compilación correcta, sin advertencias. |

Fuente: elaboración propia. Salida real capturada con Java 17.

**Figura 19.** Resultado de las pruebas automatizadas.

| Salida capturada   Java 17   Verificacion                                                                                    |
|------------------------------------------------------------------------------------------------------------------------------|
| RESULTADO: 57/57 verificaciones correctas.<br>Cobertura: intervalos, pilas, igualdad, pares, contenedor y límites genéricos. |

Fuente: elaboración propia. Salida real capturada con Java 17.

Código de las 57 verificaciones:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/tests/Verificacion.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/tests/Verificacion.java)

Registro textual de resultados:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/evidencias/Verificacion.txt](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/evidencias/Verificacion.txt)

La comprobación cubre intervalos, errores de límites, orden LIFO, conservación del estado, comparación con null, igualdad de pares y personas, acceso al contenedor y ejemplos con límites de tipo. Los archivos .txt y .png de evidencias permiten revisar los resultados sin ejecutar el proyecto.

### 9.3 Código complementario: igualdad de Persona

**Figura 20.** Sobrescritura de equals, hashCode y toString en Persona.

Código fuente | propuestos/Persona.java | líneas 1-28

```
package propuestos;

import java.util.Objects;

/** Persona inmutable, con igualdad por nombre y edad. */
public final class Persona {
    private final String nombre;
    private final int edad;

    public Persona(String nombre, int edad) {
        this.nombre = nombre;
        this.edad = edad;
    }

    @Override
    public boolean equals(Object objeto) {
        if (this == objeto) return true;
        if (!(objeto instanceof Persona)) return false;
        Persona otra = (Persona) objeto;
        return edad == otra.edad && Objects.equals(nombre, otra.nombre);
    }

    @Override
    public int hashCode() { return Objects.hash(nombre, edad); }

    @Override
    public String toString() { return nombre + " (" + edad + " años)"; }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/propuestos/Persona.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/propuestos/Persona.java)

Este código complementa el ejercicio propuesto 3. La comprobación de tipo impide convertir objetos incompatibles y el uso de Objects.equals admite un nombre null. La igualdad y el hash utilizan los mismos atributos. La clase final evita extender la jerarquía con criterios de igualdad distintos.

## 9.4 Ejemplos ejecutados del cuestionario

**Figura 21.** Límites de tipo, Comparable y wildcards.

Código fuente | cuestionario/EjemplosGenericos.java | líneas 1-30

```
package cuestionario;

import java.util.ArrayList;
import java.util.List;

public class EjemplosGenericos {
    public static <T extends Number> double duplicar(T numero) {
        return numero.doubleValue() * 2;
    }

    public static <T extends Comparable<T>> T maximo(T a, T b) {
        return a.compareTo(b) >= 0 ? a : b;
    }

    public static <T> T primero(List<T> lista) { return lista.get(0); }

    public static void mostrar(List<?> lista) {
        for (Object elemento : lista) System.out.print(elemento + " ");
        System.out.println();
    }

    public static double sumar(List<? extends Number> numeros) {
        double total = 0;
        for (Number numero : numeros) total += numero.doubleValue();
        return total;
    }

    public static void agregarEntero(List<? super Integer> destino) {
        destino.add(10);
    }
}
```

Fuente: elaboración propia. Representación del código fuente de esta entrega.

**Figura 22.** Ejecución de los ejemplos del cuestionario.

Salida capturada | Java 17 | EjemplosGenericos

```
duplicar(7): 14.0
maximo(4, 9): 9
maximo(Ana, Luis): Luis
Primero como Integer: 1
Wildcard ?: 1 2 3
Suma con extends: 6.0
Inserción con super: [10]
```

Fuente: elaboración propia. Salida real capturada con Java 17.

Código fuente:

[https://github.com/jhoao0/javaUcsm/blob/main/practica\\_05/src/cuestionario/EjemplosGenericos.java](https://github.com/jhoao0/javaUcsm/blob/main/practica_05/src/cuestionario/EjemplosGenericos.java)